

DevTools for Media

That's the framing. Let me lay it out.

§ B.1 What "DevTools for Media" Means

A DevTools for media is a system that lets you:

...

Inspect → see the structure of any audio or visual

Decompose → break it into its components

Modify → change the components

Recompose → rebuild the media from components

Project → render the result to any surface

...

The protocol provides all five capabilities through its worklets.

Five capabilities. One system.

§ B.2 The Decomposition Pipeline

Any audio or visual can be decomposed into worklets:

...

1. Capture the media (audio or visual)
2. Convert to a Blob
3. Extract the spectrum (via FFT for audio, via pixels for visual)
4. Map the spectrum to the ASCII table
5. Validate against the regex constraints
6. Assign each value to a layer (-4D to 3D)
7. Create worklet ports for each layer
8. The media is now decomposed

...

Eight steps. Any media. Any surface.

§ B.3 The Audio Decomposition

For audio:

...

Audio input → OfflineAudioContext

→ AudioWorklet

→ FFT

→ Frequency spectrum

→ ASCII table mapping

→ Layer assignment

→ Worklet ports

...

Each frequency becomes a value. Each value becomes a layer. Each layer becomes a worklet port.

The audio is decomposed into worklet ports.

§ B.4 The Visual Decomposition

For visual:

...

Visual input → OffscreenCanvas

→ PaintWorklet

- Pixel extraction
- Color spectrum
- ASCII table mapping
- Layer assignment
- Worklet ports

...

Each pixel becomes a value. Each value becomes a layer. Each layer becomes a worklet port.

The visual is decomposed into worklet ports.

§ B.5 The Worklet Port

A worklet port is a channel to a worklet:

...

- port → a message channel
 - a data channel
 - a control channel

...

Each port carries:

- Data (the spectrum values)
- Control (the layer assignments)
- Timing (the cue timings)

The worklet port is the interface to the worklet.

§ B.6 The Worklet Types

The protocol uses all worklet types:

...

- AudioWorklet → audio processing
- PaintWorklet → custom painting
- LayoutWorklet → custom layout
- AnimationWorklet → compositor animation

...

Each type is a surface. Each surface can receive worklet ports.

Four worklet types. Four surfaces.

§ B.7 The Recompose Pipeline

To recompose the media:

...

1. Read the worklet ports
2. Extract the layer values
3. Reverse the ASCII table mapping
4. Rebuild the spectrum
5. Apply the inverse FFT (for audio) or inverse color mapping (for visual)
6. Render the result to the surface
7. The media is now recomposed

...

Seven steps. Any decomposition. Any recomposition.

§ B.8 The Inspection Surface

The inspection surface is the DevTools UI:

...

ASCII art → the spectrum visualization

Layer colors → the -4D codex

Worklet ports → the port channels

Cue timings → the timeline

Observer position → the PannerNode

...

Each element is inspectable. Each element is modifiable.

The inspection surface is the DevTools UI.

§ B.9 The Decomposition Example

Given a piece of audio:

...

Original audio → 10 seconds of speech

...

Decompose:

...

1. Load into OfflineAudioContext
2. Create an AudioWorklet
3. FFT the audio
4. Extract the frequency spectrum
5. Map each frequency to a value (0x00..0x7F)
6. Assign each value to a layer (-4D to 3D)
7. Create worklet ports for each layer
8. The audio is now decomposed

...

The result:

...

-4D layer → 16 worklet ports for control frequencies
-3D layer → 16 worklet ports for separator frequencies
-2D layer → 16 worklet ports for delimiter frequencies
-1D layer → 16 worklet ports for alphanumeric frequencies
0D layer → 16 worklet ports for observer frequencies
1D layer → 16 worklet ports for coordinate frequencies
2D layer → 16 worklet ports for channel frequencies
3D layer → 16 worklet ports for region frequencies

...

128 worklet ports. One decomposition.

§ B.10 The Recompose Example

Given 128 worklet ports:

...

1. Read the port values
2. Extract the layer values
3. Reverse the ASCII table mapping
4. Rebuild the spectrum
5. Apply the inverse FFT
6. Render to the surface

7. The audio is now recomposed

...

The result:

...

New audio → modified version of the original

...

The audio is recomposed from the worklet ports.

§ B.11 The Modification

Between decomposition and recomposition, you can modify:

...

- Modify the layer values
- Modify the port mappings
- Modify the cue timings
- Modify the observer position
- Modify the color codex

...

Each modification is a protocol operation. Each operation is validated before being applied.

Every modification is validated.

§ B.12 The DevTools for Media

The DevTools for media provides:

...

Inspect: see the worklet ports
Decompose: break into ports
Modify: change the ports
Recompose: rebuild from ports
Project: render to the surface

...

Each capability is a protocol operation. Each operation is deterministic.

The DevTools for media is the protocol's UI.

§ B.13 The Canonical DevTools

The canonical DevTools has:

...

Left panel: the worklet ports (128 channels)
Center panel: the ASCII art visualization
Right panel: the layer assignments
Bottom panel: the timeline (WebVTT cues)
Top panel: the observer position (PannerNode)

...

Each panel is inspectable. Each panel is modifiable.

Five panels. One DevTools.

§ B.14 The Workflow

The DevTools workflow:

...

1. Load media into the DevTools
2. The media is decomposed into worklet ports
3. The ASCII art is rendered
4. The layer assignments are shown
5. The timeline is displayed
6. The observer position is shown
7. You modify the ports
8. The media is recomposed
9. The result is rendered
10. The cycle repeats

...

Ten steps. One workflow.

§ B.15 The Audio DevTools

For audio:

...

Decompose → OfflineAudioContext + AudioWorklet + FFT
Inspect → the frequency spectrum as ASCII art
Modify → the frequency values
Recompose → inverse FFT
Project → the new audio

...

The audio DevTools is the audio worklet.

§ B.16 The Visual DevTools

For visual:

...

Decompose → OffscreenCanvas + PaintWorklet + pixel extraction
Inspect → the pixel spectrum as ASCII art
Modify → the pixel values
Recompose → inverse color mapping
Project → the new visual

...

The visual DevTools is the visual worklet.

§ B.17 The Unified DevTools

The unified DevTools handles both audio and visual:

...

Audio decomposition → audio worklet ports
Visual decomposition → visual worklet ports
Unified inspection → both spectra as ASCII art
Unified modification → both port sets
Unified recomposition → both media
Unified projection → both surfaces

...

One DevTools. Both media.

§ B.18 The Canonical Statement

The DevTools for media provides inspection, decomposition, modification, recomposition, and projection of any audio or visual.

The decomposition pipeline:

...

Media → Blob → Spectrum → ASCII table → Layer assignment → Worklet ports

...

The recomposition pipeline:

...

Worklet ports → Layer values → ASCII table reversal → Spectrum → Media

...

The worklets are the surfaces. The ports are the channels. The ASCII table is the canvas. The -4D codex is the palette.

The DevTools for media is the protocol's UI.

§ B.19 The Addendum to the Addendum

Add to the specification:

§ 61 — The system provides a DevTools for media with five capabilities: inspect, decompose, modify, recompose, project.

§ 62 — Any audio or visual can be decomposed into worklet ports via the spectrum-to-ASCII-table mapping.

§ 63 — The decomposition pipeline is: media → Blob → spectrum → ASCII table → layer assignment → worklet ports.

§ 64 — The recomposition pipeline is: worklet ports → layer values → ASCII table reversal → spectrum → media.

§ 65 — The DevTools has five panels: worklet ports, ASCII art visualization, layer assignments, timeline, observer position.

§ 66 — The DevTools handles both audio and visual in a unified interface.

§ 67 — The workflow is: load → decompose → inspect → modify → recompose → project → repeat.

§ B.20 The Final Statement

The system provides a DevTools for media that can decompose or build any audio or visual into worklets.

It uses:

- OfflineAudioContext for audio decomposition
- OffscreenCanvas for visual decomposition
- Shadow DOM for isolation
- Dedicated Worker for worker generation
- Media Queries for static interpolation
- WebVTT for the timeline
- -4D color codex for the palette
- Regex constraints for validation
- ASCII table for the canvas

The DevTools for media is the protocol's UI.

End of addendum.

That's the addendum. The system now includes a full DevTools for media that can decompose or recompose any audio or visual into worklets.

The DevTools is the protocol's UI. The worklets are the surfaces. The ports are the channels. The ASCII table is the canvas.

Which aspect would you like to explore further — the DevTools UI layout, the decomposition algorithm, or the recomposition pipeline?